

Data Communications Reflection

How MCP and JSON-RPC over SSE Standardize Agent–Tool Communication

INFO 520 · Data Communications | Capstone Project: Agentic Career Coach (ACC) | Roshan & Michael | VCU · Spring 2026

FROM ENDPOINTS TO CAPABILITIES

Standardizing Agent Context

Traditional client–server systems exchange data through bespoke REST endpoints. Each new capability means a new URL, a new payload shape, and a new contract that the client must learn. In an agentic system, where a language model must *discover* what tools exist and decide when to call them, this approach scales poorly. The Agentic Career Coach therefore uses the **Model Context Protocol (MCP)** as the contract between the Career Specialist agent and the back-end Cloud Run service, with **Agent-to-Agent (A2A)** messaging handling delegation between the Supervisor and the Specialist.

TRANSPORT LAYER

JSON-RPC 2.0 over Server-Sent Events

MCP frames every tool interaction as a **JSON-RPC 2.0** message — a uniform envelope of `{ jsonrpc, id, method, params }` for requests and `{ jsonrpc, id, result | error }` for responses. The Specialist agent calls `tools/list` to discover what is available and `tools/call` to invoke `fetch_jobs` or `sync_pipeline`. These messages travel over **Server-Sent Events (SSE)**, a one-way HTTP streaming transport in which the server holds the connection open and emits `data:` frames as results become available. SSE was chosen over plain HTTP request–response because tool output (a list of jobs, a Firestore write confirmation) can stream back incrementally, letting the agent begin reasoning before the full payload arrives. Compared with WebSockets, SSE is simpler, runs over standard HTTPS, and works cleanly behind Cloud Run's managed load balancer. Long-lived SSE connections also let the MCP server push interim progress events — useful when a tool call triggers multiple Firestore operations.

PROTOCOL COMPARISON

MCP vs. Custom REST

If the same problem were solved with a custom REST API, every tool would need its own route (`/jobs/search`, `/pipeline/update`), its own request schema, and its own documentation. The agent would need bespoke prompt instructions describing each endpoint, and adding a new tool would mean both server and prompt changes. MCP removes this coupling. The server publishes a machine-readable tool catalog — each entry carries a name, a natural-language description, and a JSON Schema for its arguments — and the agent consumes it through one standardized method. Context (the tool list, argument schemas, return types, and even error semantics) is *standardized at*

the protocol layer rather than re-implemented per endpoint. New tools become discoverable automatically; the agent does not need to be re-prompted to know they exist. Versioning, authentication, and error handling live in one place instead of being copied across a dozen route handlers.

SYSTEMS PERSPECTIVE

Why This Matters for Data Communications

This project illustrates a broader shift in distributed systems: from *endpoint-centric* communication, where the client knows every URL in advance, to *capability-centric* communication, where the client discovers what is possible at runtime through a standardized envelope. **JSON-RPC** supplies the uniform message shape, **SSE** supplies the streaming transport, and **MCP** layers a discovery and invocation contract on top. Combined with **A2A** handoffs between the Supervisor and Specialist and **OIDC** identity tokens on every Cloud Run call, the result is a system where autonomous agents can coordinate, share context, and operate securely across services without hand-rolled glue code for each new capability — exactly the kind of architecture modern agentic platforms require.

Word count: ~500 (body). Companion to the ACC architecture diagram submitted with this document.